

# Bi-Weekly Progress Reports

## Correct but Fragile: Numerical Stability Risks in Machine Learning Systems

Cordell Stonecipher  
CSI 4150 – AI for IT Operations  
April 17, 2026

---

### Bi-Weekly Report 5 (Week 9 – Milestone 5)

---

#### Objective

Complete the end-to-end reliability pipeline and finalize the project’s experimental framework. This milestone closes the loop on the validation campaign by consolidating completed CPU seed, batch, and preprocessing experiments, adding gradient norm tracking to the reporting workflow, and incorporating a CUDA precision comparison into the final artifact set.

#### Implemented Components

- Completed the final experiment matrix across:
  - Seed variation
  - Batch size variation
  - Preprocessing-order variation
  - Gradient norm logging and summary generation
  - CUDA precision comparison for `fp32` versus `amp`
- Generated final plots and summary artifacts from the comparison CSV outputs.
- Finalized the complete operational pipeline:
  - Reproducibility
  - Stability measurement
  - Gradient norm tracking
  - Cross-run analysis
  - CI enforcement
  - Artifact reporting and final documentation

#### Completed Validation Scope

Milestone 5 serves as the technical completion point for the project. The finalized system now supports and documents:

- Deterministic CPU baseline runs
- Preprocessing-order sweep with artifact-backed disagreement comparison
- Gradient norm logging across training runs

- CUDA-backed precision comparison for `fp32` and `amp`
- Prediction-level comparison artifacts for post-hoc reproducibility analysis
- CI threshold checks that turn instability into an enforceable engineering policy

### Evidence (Screenshots)

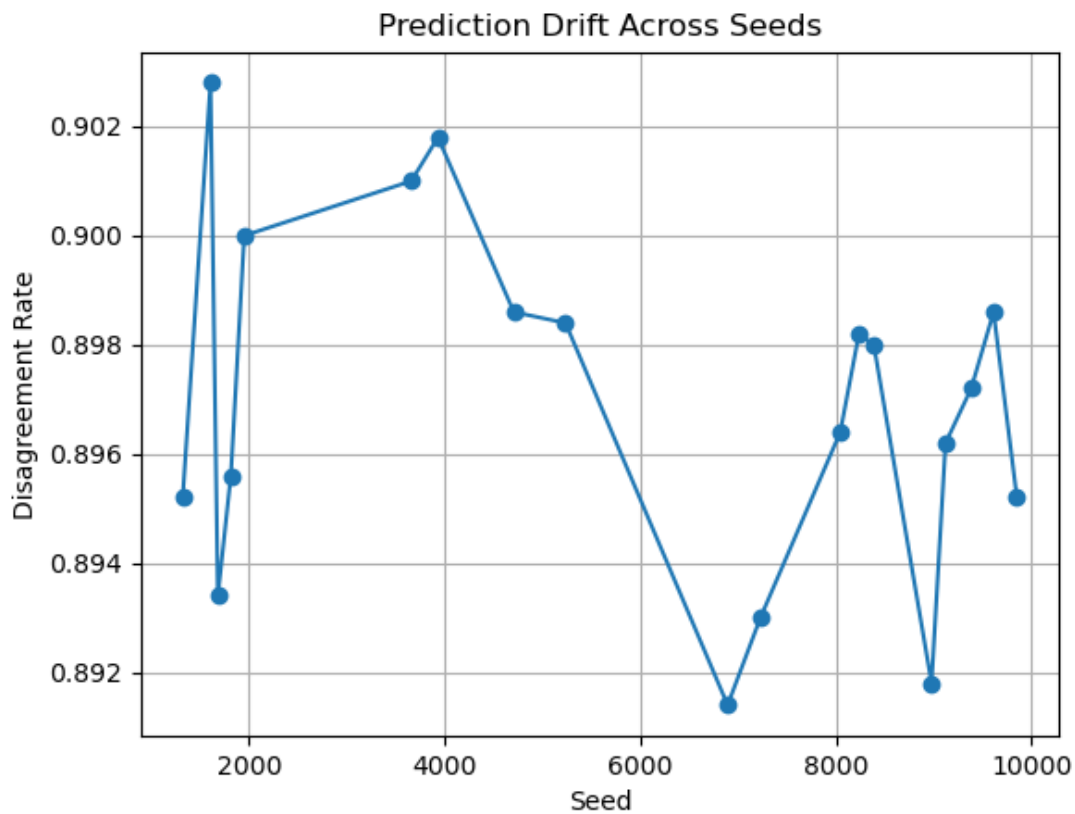


Figure 1: Prediction disagreement rate across seed sweep showing variability despite similar accuracy.

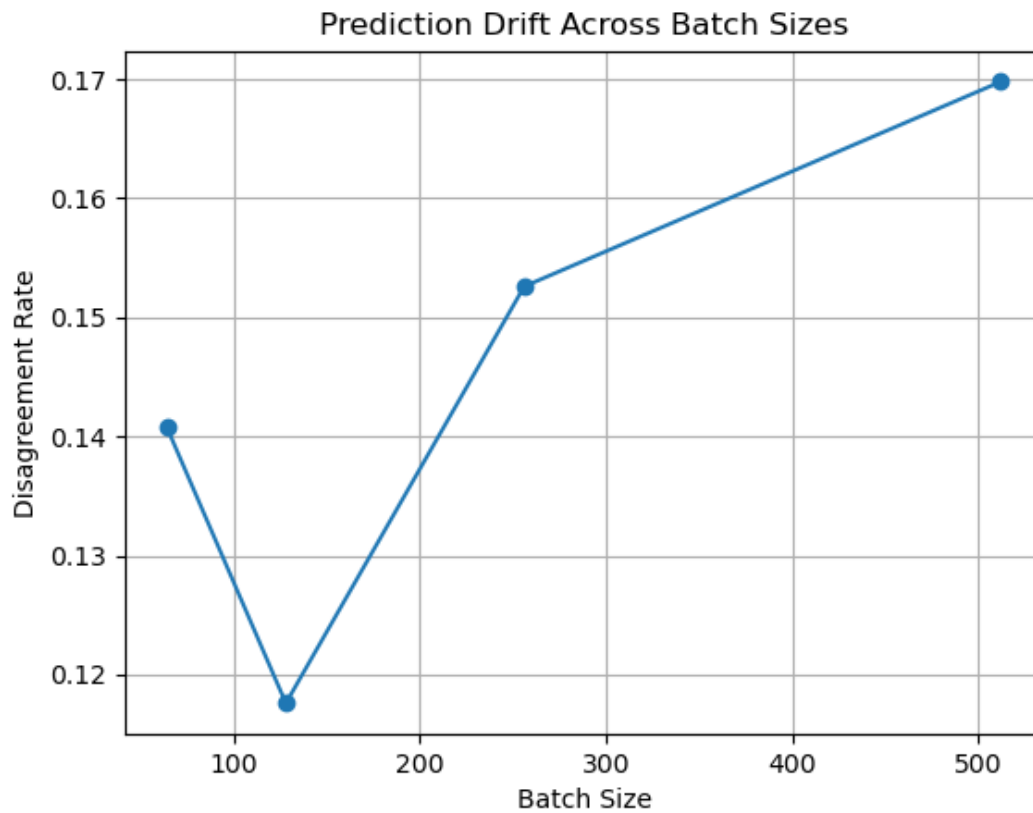


Figure 2: Prediction disagreement rate across batch sizes demonstrating sensitivity to training configuration.

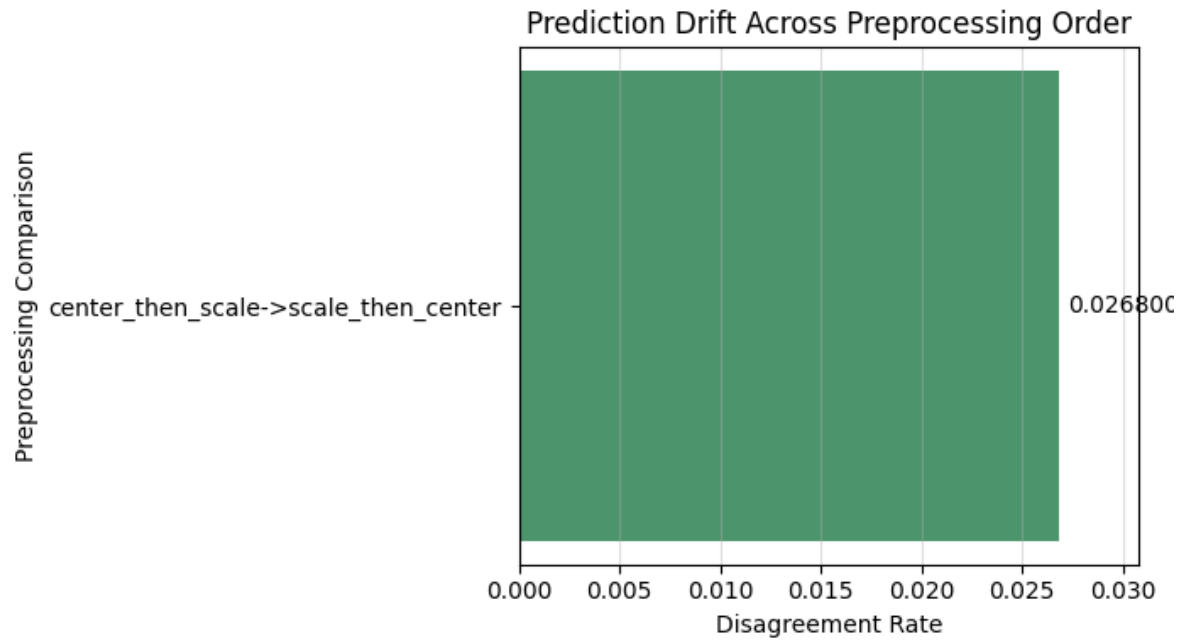


Figure 3: Prediction disagreement across preprocessing-order variants showing that input normalization order can change final predictions.

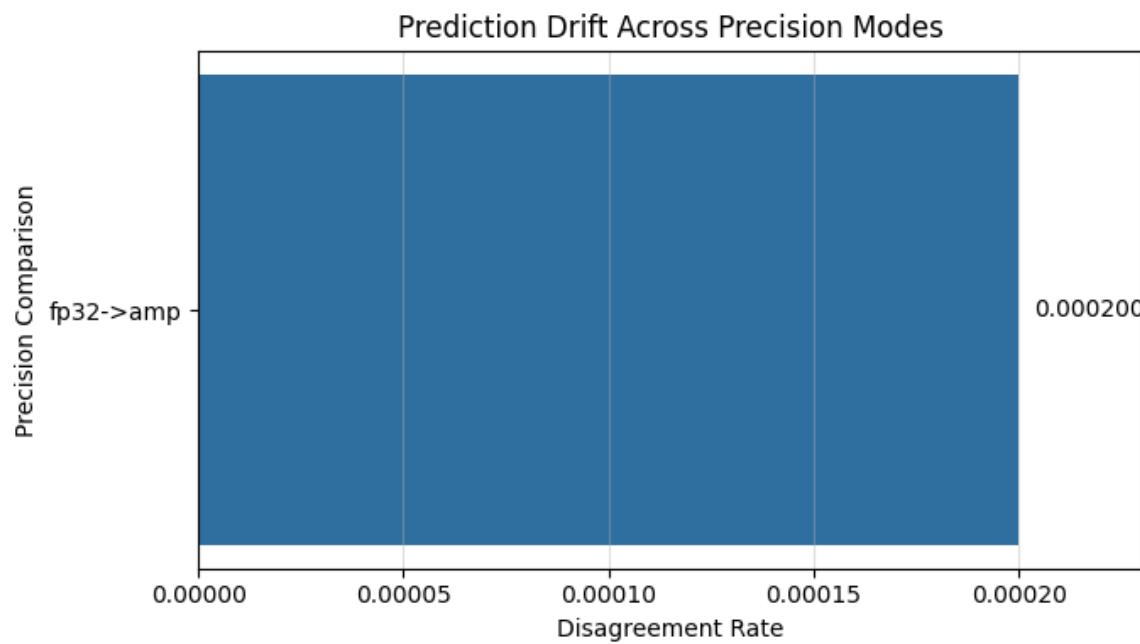


Figure 4: Prediction disagreement between CUDA `fp32` and `amp` runs, showing that precision mode can change outputs even when aggregate accuracy is stable.

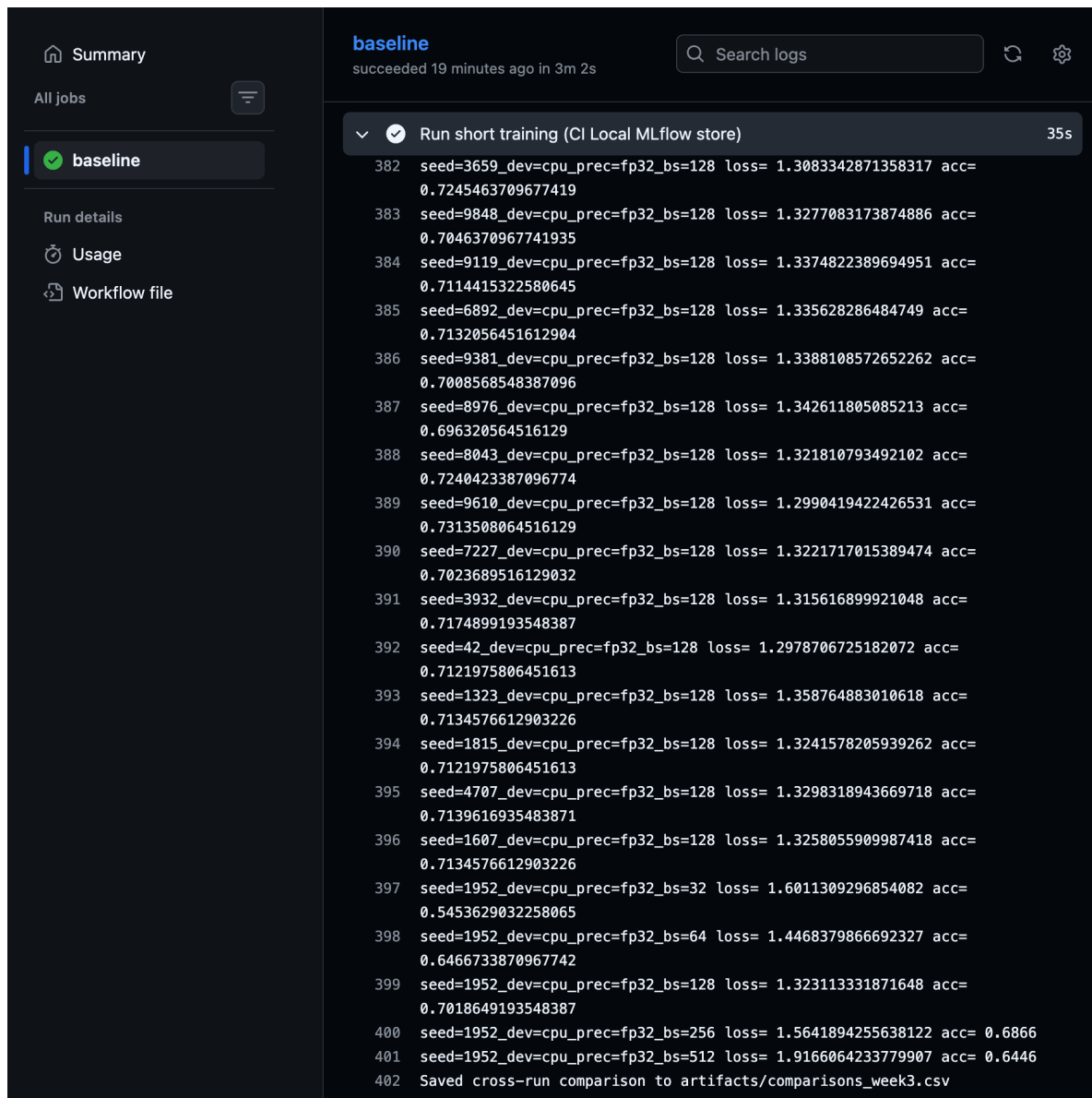


Figure 5: Successful CI execution showing the completed reliability pipeline operating with stability checks enabled.

## Key Findings

- Prediction outputs vary across seeds and batch sizes even when accuracy remains stable.
- Preprocessing order introduces measurable prediction drift even under deterministic execution.
- Precision mode should be treated as a reproducibility variable, not an implementation detail.
- Gradient norm logging adds visibility into training-time numerical behavior beyond final accuracy.
- Numerical instability is measurable and reproducible.
- Perturbation-based instability provides a reliable CI gating signal.

## Final System Capabilities

- Deterministic reproducible training pipeline
- Numerical stability measurement (within-run)
- Gradient norm telemetry during training
- Cross-run reproducibility analysis
- Artifact-backed experiment tracking
- CPU sweep execution plus CUDA precision-aware evaluation
- CI-based stability enforcement
- Final reporting in both milestone and IEEE formats

## Conclusion

Milestone 5 represents the technical completion of the project’s planned reliability workflow. The final repository artifact set focuses on the completed CPU-based seed, batch-size, and preprocessing-order experiments, along with a targeted CUDA precision comparison, establishing a complete end-to-end demonstration of stability-aware MLOps validation.

## Future Work

Future work includes expanding the evaluation to additional hardware environments, broadening precision studies beyond `fp32` and `amp`, integrating stability metadata into model registry workflows, and building automated alerting and monitoring dashboards.

## Bi-Weekly Report 4 (Week 7 – Milestone 4)

---

### Objective

Introduce automated reliability enforcement into the pipeline by converting numerical instability metrics into CI failure conditions. This milestone focuses on validating model stability under small perturbations while preserving cross-run variability as an experimental signal.

### Implemented Components

- Introduced environment-configurable threshold:
  - `MAX_PERTURB_DISAGREE` for within-run instability
- Implemented **within-run stability gating**:
  - Training fails if perturbation disagreement exceeds threshold.
- Integrated threshold enforcement into GitHub Actions CI pipeline.
- Retained cross-run disagreement as a **measured artifact**, not a failure condition.

## Gating Logic

Within-run condition:

Fail if  $\text{Disagree}_\epsilon > \tau_{\text{perturb}}$

Where  $\tau_{\text{perturb}}$  is defined via environment variables.

Evidence (Screenshots)

```
env:
  MLFLOW_TRACKING_URI: "sqlite:///mlflow_ci.db"
  MAX_PERTURB_DISAGREE: "0.05"
```

Figure 6: Environment variable configuration for perturbation stability threshold in local and CI environments.

```
rical-fragility-mlops/src/train.py", line 209, in train_one
    raise RuntimeError(
RuntimeError: [FAIL] Perturbation instability too high: 0.0006 > 0.0001
```

Figure 7: Forced failure demonstrating within-run instability gate triggering when perturbation disagreement exceeds threshold.

```
(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops % export MAX_PERTURB_DISAGREE=0.05
(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops % python src/train.py
seed=1952_dev=cpu_prec=fp32_bs=128 loss= 1.3231133434080309 acc= 0.7018649193548387
seed=5235_dev=cpu_prec=fp32_bs=128 loss= 1.3167029580762308 acc= 0.7111895161290323
seed=8234_dev=cpu_prec=fp32_bs=128 loss= 1.3295662306970166 acc= 0.7338709677419355
seed=8386_dev=cpu_prec=fp32_bs=128 loss= 1.3495271282811319 acc= 0.7111895161290323
seed=1682_dev=cpu_prec=fp32_bs=128 loss= 1.3067553101047393 acc= 0.7106854838709677
seed=3659_dev=cpu_prec=fp32_bs=128 loss= 1.3083342948267538 acc= 0.7245463709677419
seed=9848_dev=cpu_prec=fp32_bs=128 loss= 1.3277083154647582 acc= 0.7046370967741935
seed=9119_dev=cpu_prec=fp32_bs=128 loss= 1.3374822332013039 acc= 0.7114415322580645
seed=6892_dev=cpu_prec=fp32_bs=128 loss= 1.3356282826392882 acc= 0.7132056451612904
seed=9381_dev=cpu_prec=fp32_bs=128 loss= 1.3388108553424958 acc= 0.7008568548387096
seed=8976_dev=cpu_prec=fp32_bs=128 loss= 1.342611808930674 acc= 0.696320564516129
seed=8043_dev=cpu_prec=fp32_bs=128 loss= 1.321810789646641 acc= 0.7240423387096774
seed=9610_dev=cpu_prec=fp32_bs=128 loss= 1.2990419403199227 acc= 0.7313508064516129
seed=7227_dev=cpu_prec=fp32_bs=128 loss= 1.3221717111525997 acc= 0.7023689516129032
seed=3932_dev=cpu_prec=fp32_bs=128 loss= 1.315616899921048 acc= 0.7174899193548387
seed=42_dev=cpu_prec=fp32_bs=128 loss= 1.2978676961314293 acc= 0.7121975806451613
seed=1323_dev=cpu_prec=fp32_bs=128 loss= 1.3587648676287742 acc= 0.7134576612903226
seed=1815_dev=cpu_prec=fp32_bs=128 loss= 1.3241578225166566 acc= 0.7121975806451613
seed=4707_dev=cpu_prec=fp32_bs=128 loss= 1.3298318520669015 acc= 0.7139616935483871
seed=1607_dev=cpu_prec=fp32_bs=128 loss= 1.3258055660032457 acc= 0.7134576612903226
seed=1952_dev=cpu_prec=fp32_bs=32 loss= 1.6011309431445213 acc= 0.5453629032258065
seed=1952_dev=cpu_prec=fp32_bs=64 loss= 1.446838019355651 acc= 0.6466733870967742
seed=1952_dev=cpu_prec=fp32_bs=128 loss= 1.3231133434080309 acc= 0.7018649193548387
seed=1952_dev=cpu_prec=fp32_bs=256 loss= 1.564189443874359 acc= 0.6866
seed=1952_dev=cpu_prec=fp32_bs=512 loss= 1.9166064291000366 acc= 0.6446
Saved cross-run comparison to artifacts/comparisons_week4.csv
```

Figure 8: Cross-run comparison retained as an analysis artifact showing disagreement across seed and batch sweeps.

```

409   File "/home/runner/work/numerical-fragility-mlops/numerical-fragility-
      mlops/src/train.py", line 296, in compute_cross_run_disagreement
410       raise RuntimeError(
411   RuntimeError: [FAIL] Max cross-run instability 0.9028 exceeds threshold 0.05
412   Error: Process completed with exit code 1.

```

Figure 9: GitHub Actions CI failure when perturbation instability threshold is violated.

The screenshot shows the GitHub Actions interface for a workflow named 'baseline'. The job 'Run short training (CI Local MLflow store)' is shown as successful, having completed 35 seconds ago. The log output lists 20 individual training runs, each with a seed value, device (cpu), precision (fp32), batch size (bs), loss, and accuracy (acc). The runs are numbered 382 through 402. The final line of the log indicates that a cross-run comparison was saved to a file named 'comparisons\_week3.csv'.

Figure 10: Successful CI run when stability remains within acceptable limits.

## Key Outcome

Milestone 4 introduces enforceable reliability constraints into the ML pipeline. Numerical instability is no longer just observed—it directly determines whether a run is considered valid within CI.

## Interpretation

- Within-run perturbation sensitivity provides a principled measure of numerical stability.
- Cross-run disagreement remains an experimental metric reflecting intended configuration variation.
- Separating enforcement (CI gating) from experimentation (sweep analysis) ensures correct interpretation of results.

## Impact

This milestone aligns the project with production MLOps practices by enforcing reliability constraints during automated validation while preserving flexibility for experimental analysis.

## Next Steps (Finalization)

- Add visualization of instability trends (seed vs batch plots).
- Finalize report and presentation materials.

## Bi-Weekly Report 3 (Week 5 – Milestone 3)

---

### Objective

Extend the numerical stability framework to quantify cross-run prediction drift across operational perturbations. Milestone 3 introduces run-to-run disagreement analysis for both seed sweeps and batch size sweeps using saved prediction artifacts.

### Implemented Components

- Added cross-run comparison logic to compute disagreement rates between prediction snapshots.
- Grouped runs by sweep type:
  - **Seed sweep:** fixed batch size, varying random seeds.
  - **Batch sweep:** fixed seed, varying batch sizes.
- Selected deterministic baseline per sweep:
  - Lowest seed for seed sweep.
  - Smallest batch size for batch sweep.
- Computed disagreement rate:

$$\text{Disagreement} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i^{(baseline)} \neq \hat{y}_i^{(compare)})$$

- Exported results to `artifacts/comparisons_week2.csv`.
- Logged comparison CSV to MLflow as an experiment artifact.

## Evidence (Screenshots)

```
artifacts > comparisons_week2.csv > data
```

|    | sweep_type  | baseline_hash | compare_hash | baseline_seed | compare_seed | baseline_batch | compare_batch | disagreement_rate |
|----|-------------|---------------|--------------|---------------|--------------|----------------|---------------|-------------------|
| 1  | seed_sweep  | e319adf3cffd  | b940680b14fd | 42            | 1323         | 128            | 128           | 0.8952            |
| 2  | seed_sweep  | e319adf3cffd  | 97b115029002 | 42            | 1607         | 128            | 128           | 0.9028            |
| 3  | seed_sweep  | e319adf3cffd  | 07854ced331f | 42            | 1682         | 128            | 128           | 0.8934            |
| 4  | seed_sweep  | e319adf3cffd  | b9a9d403022c | 42            | 1815         | 128            | 128           | 0.8956            |
| 5  | seed_sweep  | e319adf3cffd  | 5a02fbb39017 | 42            | 1952         | 128            | 128           | 0.9               |
| 6  | seed_sweep  | e319adf3cffd  | 53831dc125c8 | 42            | 3659         | 128            | 128           | 0.901             |
| 7  | seed_sweep  | e319adf3cffd  | 6a94cf22c58d | 42            | 3932         | 128            | 128           | 0.9018            |
| 8  | seed_sweep  | e319adf3cffd  | d81484779a81 | 42            | 4707         | 128            | 128           | 0.8986            |
| 9  | seed_sweep  | e319adf3cffd  | 533b65205d6c | 42            | 5235         | 128            | 128           | 0.8984            |
| 10 | seed_sweep  | e319adf3cffd  | 9188d3abf580 | 42            | 6892         | 128            | 128           | 0.8914            |
| 11 | seed_sweep  | e319adf3cffd  | a7ca8084d3ee | 42            | 7227         | 128            | 128           | 0.893             |
| 12 | seed_sweep  | e319adf3cffd  | e9f2cdc0b8c6 | 42            | 8043         | 128            | 128           | 0.8964            |
| 13 | seed_sweep  | e319adf3cffd  | ff61d3b96555 | 42            | 8234         | 128            | 128           | 0.8982            |
| 14 | seed_sweep  | e319adf3cffd  | c1efba00b3a8 | 42            | 8386         | 128            | 128           | 0.898             |
| 15 | seed_sweep  | e319adf3cffd  | 77d47e2e09ef | 42            | 8976         | 128            | 128           | 0.8918            |
| 16 | seed_sweep  | e319adf3cffd  | 6fd72d45b873 | 42            | 9119         | 128            | 128           | 0.8962            |
| 17 | seed_sweep  | e319adf3cffd  | b417091249cb | 42            | 9381         | 128            | 128           | 0.8972            |
| 18 | seed_sweep  | e319adf3cffd  | 3d58f35f706a | 42            | 9610         | 128            | 128           | 0.8986            |
| 19 | seed_sweep  | e319adf3cffd  | 446aef850946 | 42            | 9848         | 128            | 128           | 0.8952            |
| 20 | batch_sweep | 800ef09af40c  | eca07a0b2972 | 1952          | 1952         | 32             | 64            | 0.1408            |
| 21 | batch_sweep | 800ef09af40c  | 3c629c124d21 | 1952          | 1952         | 32             | 128           | 0.1176            |
| 22 | batch_sweep | 800ef09af40c  | 6a6f94740466 | 1952          | 1952         | 32             | 256           | 0.1526            |
| 23 | batch_sweep | 800ef09af40c  | a9141ac72a6d | 1952          | 1952         | 32             | 512           | 0.1698            |

Figure 11: Generated comparisons.csv showing baseline-to-run disagreement rates across sweeps.

Previewing the first 23 rows

| sweep_type | baseline_hash | compare_hash | baseline_seed | compare_seed | baseline_batch | compare_batch | disagreement_rate |
|------------|---------------|--------------|---------------|--------------|----------------|---------------|-------------------|
| seed_sweep | e319adf3cffd  | b940680b14fd | 42            | 1323         | 128            | 128           | 0.8952            |
| seed_sweep | e319adf3cffd  | 97b115029002 | 42            | 1607         | 128            | 128           | 0.9028            |
| seed_sweep | e319adf3cffd  | 07854ced331f | 42            | 1682         | 128            | 128           | 0.8934            |
| seed_sweep | e319adf3cffd  | b9a9d403022c | 42            | 1815         | 128            | 128           | 0.8956            |
| seed_sweep | e319adf3cffd  | 5a02fbb39017 | 42            | 1952         | 128            | 128           | 0.9               |
| seed_sweep | e319adf3cffd  | 53831dc125c8 | 42            | 3659         | 128            | 128           | 0.901             |
| seed_sweep | e319adf3cffd  | 6a94cf22c58d | 42            | 3932         | 128            | 128           | 0.9018            |

Figure 12: MLflow artifact view confirming comparison CSV logged under experiment runs.

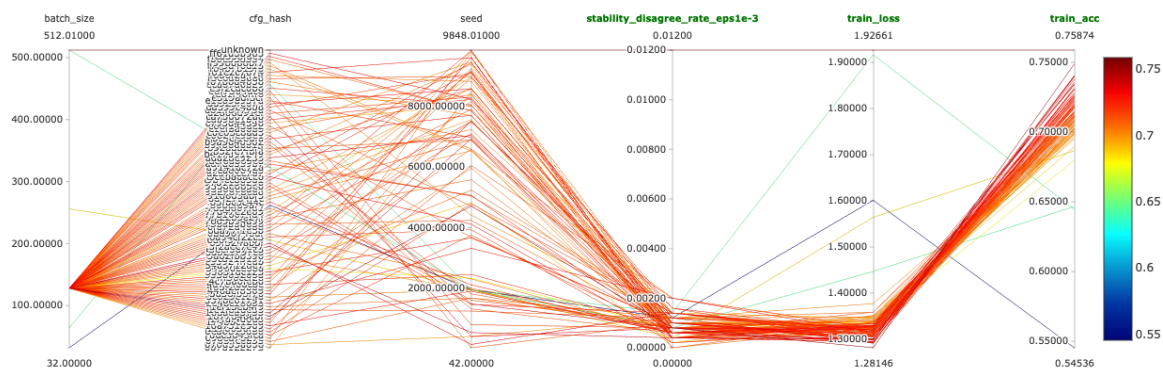


Figure 13: MLflow comparison interface displaying stability metrics across seed and batch sweeps.

## Key Outcome

Milestone 3 establishes measurable cross-run reproducibility gaps. Even when accuracy remains stable, prediction-level disagreement is observed across seeds and batch sizes. This confirms the central thesis: correctness metrics alone do not capture operational fragility.

## Interpretation

- Seed variation introduces measurable but bounded prediction drift.
- Batch size variation demonstrates additional sensitivity in output consistency.
- Disagreement rates provide a more granular stability signal than accuracy alone.

## Next Bi-Weekly Plan

- Introduce CI stability thresholds (fail build if disagreement exceeds limit).
- Formalize stability gating logic using environment variables.
- Compare CPU vs GPU and FP32 vs AMP precision modes.
- Produce visual stability comparison plots for final report.

## Bi-Weekly Report 2 (Week 3 – Milestone 2)

---

### Objective

Extend the Week 1 reproducibility baseline by adding (1) a larger operational configuration sweep (randomized seed sweep + expanded batch size sweep) and (2) prediction snapshot artifacts per run. This milestone is intended to enable run-to-run agreement analysis in the next milestone.

### Implemented Components (Milestone 2)

- **Expanded configuration matrix:** randomized (but reproducible) seed sweep and expanded batch size sweep to simulate operational perturbations.
- **Stability evaluation function:** added `stability_eval` to compute perturbation sensitivity using  $\epsilon = 10^{-3}$ , logging:
  - Prediction disagreement rate between  $x$  and  $x + \epsilon$
  - Mean logit variance under perturbation
- **Prediction snapshot artifacts per run:** for each configuration, the pipeline now performs inference on a fixed evaluation subset and stores:
  - `pred.npy` (predicted class indices)
  - `logits.npy` (raw logits)
  - `summary.json` (config + key metrics)
- **MLflow artifact logging:** per-run prediction snapshots are logged into MLflow under `predictions/{cfg_hash}` for later audit and comparisons.

### Evidence (Screenshots)

#### Configuration Matrix (Week 2)

Listing 1: Week 2 Configuration Matrix (`src/config.py`)

```
# src/config.py
import random
# -----
# Controlled randomness
# -----
# This ensures that the randomly generated seeds
# are reproducible across runs of the config file.
random.seed(2026)

NUM_SEEDS = 100
SEEDS = random.sample(range(1, 10000), NUM_SEEDS)

# Expanded batch size sweep
BATCH_SIZES = [32, 64, 128, 256, 512]

DEVICE = "cpu"
PRECISION = "fp32"

CONFIG_MATRIX = []
```

```

# -----
# Seed sweep (fixed batch size)
# -----
for s in SEEDS:
    CONFIG_MATRIX.append(
        {
            "tag": "seed_sweep",
            "seed": s,
            "device": DEVICE,
            "precision": PRECISION,
            "batch_size": 128,
        }
    )

# -----
# Batch size sweep (fixed seed)
# -----
BASELINE_SEED = SEEDS[0]

for bs in BATCH_SIZES:
    CONFIG_MATRIX.append(
        {
            "tag": "batch_sweep",
            "seed": BASELINE_SEED,
            "device": DEVICE,
            "precision": PRECISION,
            "batch_size": bs,
        }
    )

if __name__ == "__main__":
    print("Generated SEEDS:", SEEDS)
    print("Total configs:", len(CONFIG_MATRIX))

```

## Design Rationale

The configuration matrix separates operational perturbations into two controlled sweeps:

- **Seed sweep:** evaluates stochastic training variance under identical hyperparameters.
- **Batch size sweep:** evaluates numerical sensitivity to gradient accumulation and memory behavior.

This structure enables cross-run agreement analysis in the next milestone.

```

(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops % python src/config.py
Generated SEEDS: [1952, 5235, 8234, 8386, 1682, 3659, 9848, 9119, 6892, 9381, 8976, 8043, 9610,
7227, 3932, 42, 1323, 1815, 4707, 1607, 7368, 189, 8033, 5149, 3445, 6509, 4121, 5697, 5844, 616
7, 8415, 1251, 5578, 1470, 9134, 8816, 4787, 4713, 7495, 2309, 9461, 5063, 406, 6044, 5957, 7555
, 6918, 1486, 6534, 9597, 9085, 8167, 1908, 6971, 8297, 9834, 8046, 6524, 8596, 4238, 6881, 9241
, 7852, 8405, 8487, 416, 9430, 3679, 2114, 801, 8573, 1681, 7036, 7702, 2334, 7899, 3751, 2000,
7485, 7691, 8940, 4113, 9172, 6588, 5041, 8981, 8224, 7847, 6586, 7096, 3319, 4824, 4408, 152, 5
43, 2456, 9786, 7508, 8568, 8021]
Total configs: 105

```

Figure 14: Terminal output showing Week 2 training runs executing with the expanded config sweep.

```
(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops %
tree artifacts/predictions -L 2
artifacts/predictions
├── 007a12227561
│   ├── logits.npy
│   ├── pred.npy
│   └── summary.json
├── 070d55a0568e
│   ├── logits.npy
│   ├── pred.npy
│   └── summary.json
├── 07854ced331f
│   ├── logits.npy
│   ├── pred.npy
│   └── summary.json
└── 098ea7527e09
    ├── logits.npy
    ├── pred.npy
    └── summary.json
```

Figure 15: Local artifact outputs under `artifacts/predictions/{cfg_hash}/` including `pred.npy`, `logits.npy`, and `summary.json`.

seed=1952\_dev=cpu\_prec=fp32\_bs=512

Overview Model metrics System metrics Traces **Artifacts**

predictions

- a9141ac72a6d
  - logits.npy
  - pred.npy
  - summary.json

**predictions/a9141ac72a6d/summary.json** 412B

Path: /Users/cordellstonecipher/OU\_Undergrad/OU\_Undergrad/Winter2026/CSI4

```
{
  "cfg_hash": "a9141ac72a6d",
  "cfg": {
    "tag": "batch_sweep",
    "seed": 1952,
    "device": "cpu",
    "precision": "fp32",
    "batch_size": 512
  },
  "n_eval": 5000,
  "metrics": {
    "train_loss": 1.9166064291000366,
    "train_acc": 0.6446,
    "train_seconds": 0.10048317909240723,
    "stability_disagree_rate_eps1e-3": 0.0014,
    "stability_logit_var_mean_eps1e-3": 6.144289841358841e-7
  }
}
```

Figure 16: MLflow UI showing prediction snapshots logged as artifacts under `predictions/{cfg_hash}/`.

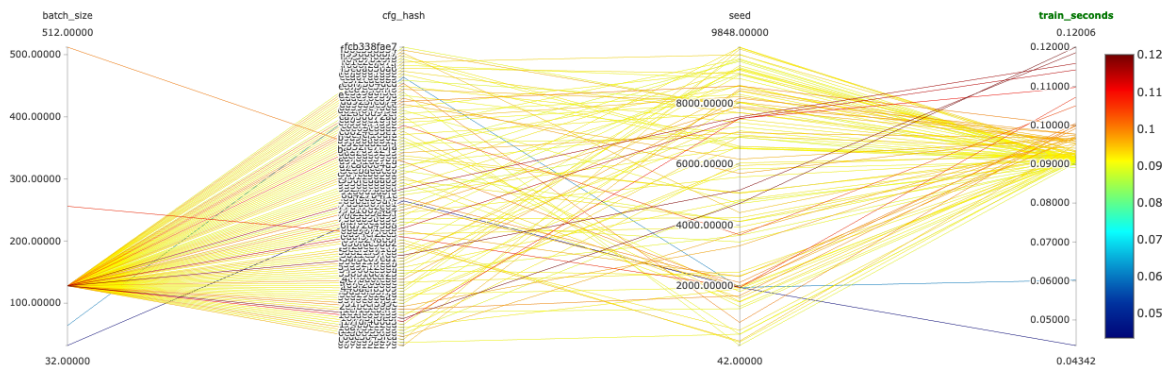


Figure 17: MLflow run comparison view showing stability metrics across configuration sweep.

## Key Outcome

Milestone 2 upgraded the pipeline from metric-only stability logging to evidence-producing runs with reproducible prediction snapshots. This makes it possible to compute and report cross-run disagreement (seed-to-seed and batch-to-batch) in Milestone 3 without re-running experiments.

## Next Bi-Weekly Plan

- Compute **cross-run agreement metrics** versus a baseline run within each sweep (seed sweep and batch sweep).
- Output a comparison artifact (CSV/JSON) summarizing disagreement rates and log it to MLflow.
- Introduce a **CI stability gate** (fail build if disagreement exceeds a threshold) using environment-configurable limits.

## Bi-Weekly Report 1 (Week 1)

---

### Objective

Implement a reproducible baseline pipeline that demonstrates numerical stability risks as an operational concern. Week 1 focused on establishing the toolchain and producing verifiable artifacts (tracking, versioning, CI, and stability metrics).

### Implemented Components

- Repository structure and modular code layout (`src/` split for model/train/config/stability).
- Deterministic baseline PyTorch training run(s) with a small fixed subset for fast iteration.
- MLflow experiment tracking (local dev) with stability-adjacent metadata logging.
- Numerical stability metrics based on perturbation sensitivity ( $\epsilon = 10^{-3}$ ):
  - Prediction disagreement rate between  $x$  and  $x + \epsilon$
  - Mean logit variance under perturbation
- DVC dataset snapshot tracking (raw data directory versioned).
- GitHub Actions workflow to run baseline training in a clean environment (CI uses a file-based MLflow store, avoiding external service dependencies).
- Dockerfile stub for environment reproducibility.

## Evidence (Screenshots)

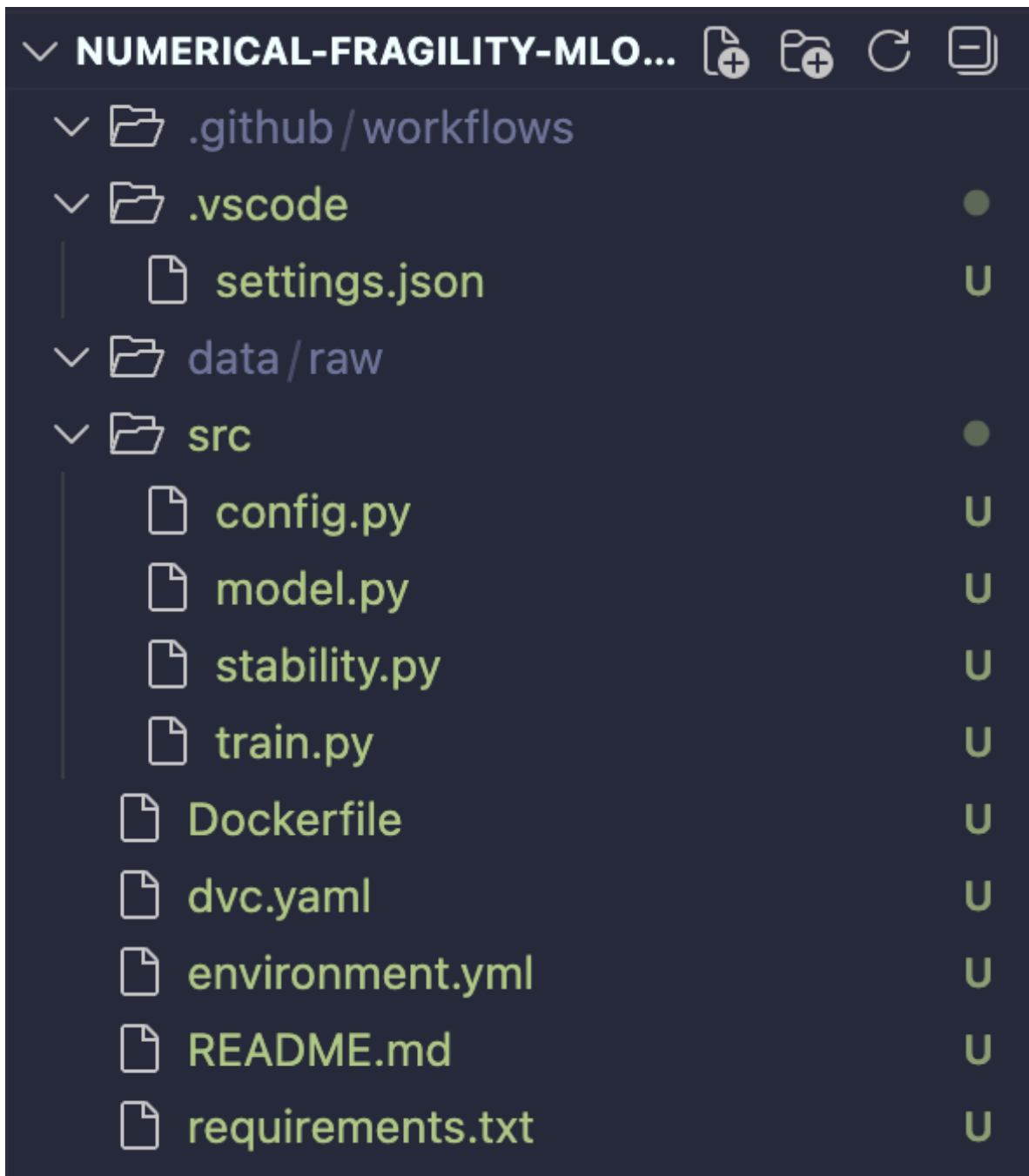
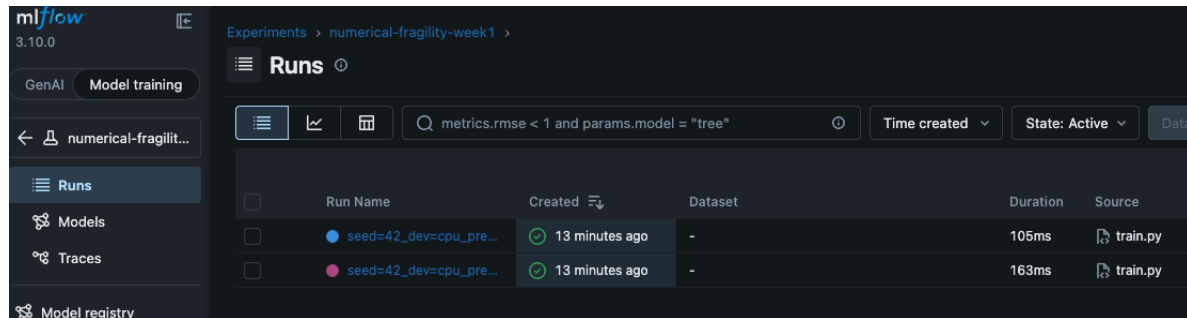


Figure 18: Repository directory structure showing project layout.

```
(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops % python -c "import torch; print(torch.__version__)"
python src/train.py
2.10.0
2026/02/22 11:02:04 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/02/22 11:02:04 INFO mlflow.store.db.utils: Updating database tables
2026/02/22 11:02:04 INFO mlflow.tracking.fluent: Experiment with name 'numerical-fragility-week1' does not exist. Creating a new experiment.
100%|          | 9.91M/9.91M [00:00<00:00, 31.3MB/s]
100%|          | 28.9k/28.9k [00:00<00:00, 802kB/s]
100%|          | 1.65M/1.65M [00:00<00:00, 7.11MB/s]
100%|          | 4.54k/4.54k [00:00<00:00, 4.63MB/s]
```

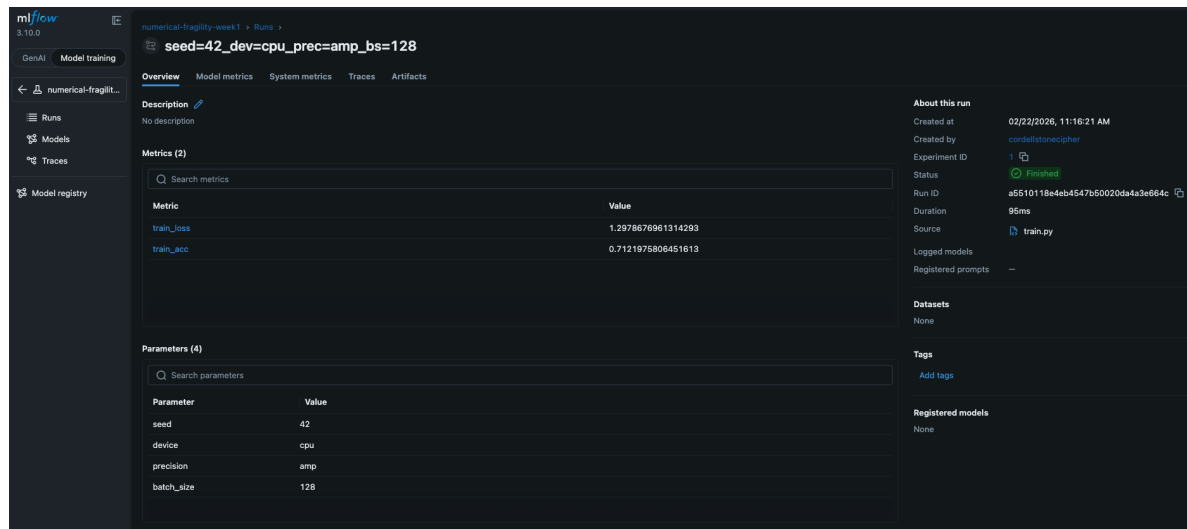
Figure 19: Terminal output showing baseline training execution across configuration variants.



The screenshot shows the MLflow UI for an experiment named 'numerical-fragility-week1'. The 'Runs' tab is active, displaying a table of runs. The table has columns for Run Name, Created, Dataset, Duration, and Source. Two runs are listed, both with a status of 'Active' and a duration of 105ms and 163ms respectively. The source for both runs is 'train.py'.

| Run Name               | Created        | Dataset | Duration | Source   |
|------------------------|----------------|---------|----------|----------|
| seed=42_dev=cpu_pre... | 13 minutes ago | -       | 105ms    | train.py |
| seed=42_dev=cpu_pre... | 13 minutes ago | -       | 163ms    | train.py |

Figure 20: MLflow UI showing tracked experiment runs and logged parameters.



The screenshot shows the MLflow UI for the 'Overview' tab of the experiment 'numerical-fragility-week1'. The 'Metrics (2)' section displays a table with two metrics: 'train\_loss' and 'train\_acc'. The 'Parameters (4)' section displays a table with four parameters: 'seed', 'device', 'precision', and 'batch\_size'. The 'About this run' section on the right provides details about the run, including the created time, created by, experiment ID, status, run ID, duration, source, logged models, and registered prompts.

| Metric     | Value              |
|------------|--------------------|
| train_loss | 1.2978676961314293 |
| train_acc  | 0.7121975806451613 |

| Parameter  | Value |
|------------|-------|
| seed       | 42    |
| device     | cpu   |
| precision  | amp   |
| batch_size | 128   |

Figure 21: MLflow metrics view showing perturbation sensitivity stability indicators.

```
(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops % git log --oneline -5
0fdda98 (HEAD -> main) Week1: log runs to MLflow server backend
(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops % dvc init
Initialized DVC repository.
```

Figure 22: DVC status confirming dataset tracking.

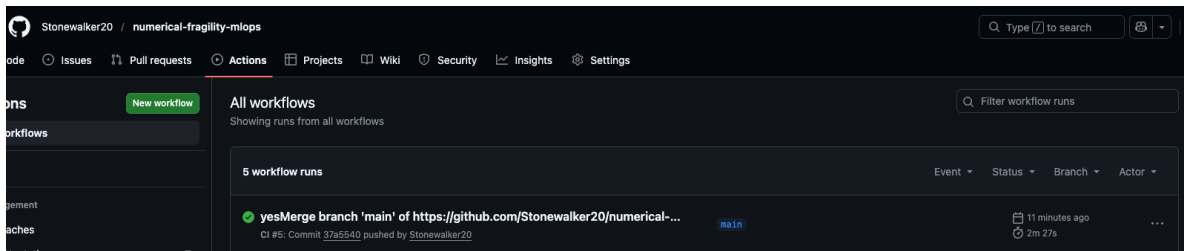


Figure 23: Successful GitHub Actions CI run validating baseline execution.

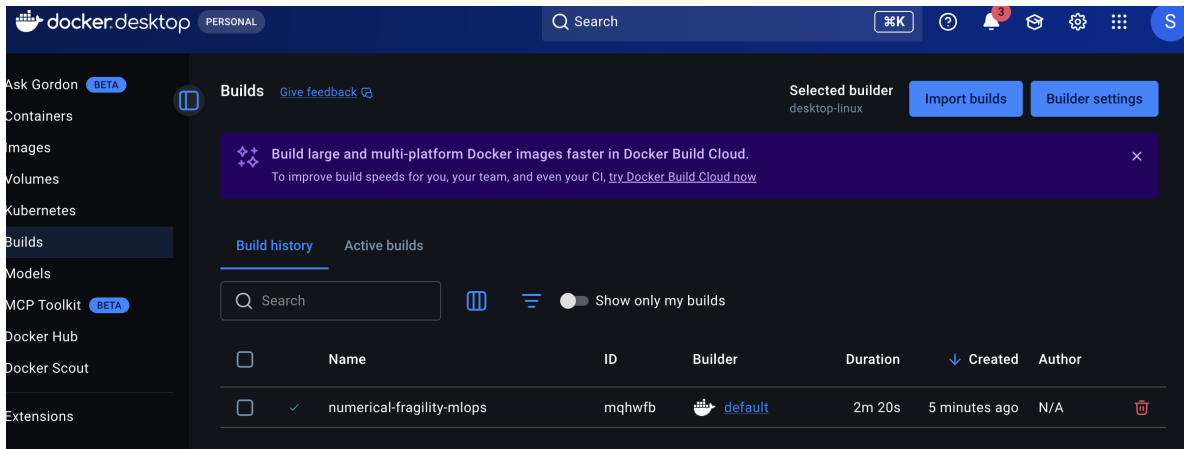


Figure 24: Docker image successfully built for reproducible execution.

```

(base) cordellstonecipher@Cordells-MacBook-Pro numerical-fragility-mlops % git log
~/OU_Undergrad/skillbridge/backend — zsh 9180824723b59a6ae6823bbb7 (HEAD -> main, origin/main)
Merge: 942f47a 9542d4f
Author: Cordell Stonecipher <cordellstonecipher@Cordells-MacBook-Pro.local>
Date: Sun Feb 22 13:17:47 2026 -0500

    yesMerge branch 'main' of https://github.com/Stonewalker20/numerical-fragility-mlops

commit 942f47a07dea73f65356c50a31554ecc7a160469
Author: Cordell Stonecipher <cordellstonecipher@Cordells-MacBook-Pro.local>
Date: Sun Feb 22 13:17:39 2026 -0500

    CI: use MLflow file store; local server via env var

commit 9542d4f956f7262d1a1a8f7e71d2f7e5afb0a555
Author: Cordell <211035440+Stonewalker20@users.noreply.github.com>
Date: Sun Feb 22 13:01:50 2026 -0500

    Remove course information from README

commit a9bc2c65c4474967bccc89d13ca53fc44316aa60
Author: Cordell Stonecipher <cordellstonecipher@Cordells-MacBook-Pro.local>
Date: Sun Feb 22 13:01:23 2026 -0500

    Week1: Updated README

commit 5c4fb8d15e382a202b6bebca6de3ab8c6477a904
Author: Cordell Stonecipher <cordellstonecipher@Cordells-MacBook-Pro.local>
Date: Sun Feb 22 12:58:19 2026 -0500

    Week1: add Dockerfile for reproducible baseline run

commit b3b7a9228edb1ad926382560a2e467e644934777
Author: Cordell Stonecipher <cordellstonecipher@Cordells-MacBook-Pro.local>
Date: Sun Feb 22 12:52:54 2026 -0500

    Week1: add CI workflow to run baseline training

commit 215d8fe7b59840b343fb73af3b52f43bd8f9990d
Author: Cordell Stonecipher <cordellstonecipher@Cordells-MacBook-Pro.local>
Date: Sun Feb 22 12:27:37 2026 -0500

```

Figure 25: Git commit history showing incremental Week 1 implementation progress.

## Key Outcome

Week 1 established a reproducible baseline that can be executed and compared across operational contexts. The pipeline now logs not only correctness metrics (loss/accuracy) but also early numerical stability indicators suitable for later CI gating and configuration-matrix expansion.

## Next Bi-Weekly Plan

- Expand configuration matrix to include seed sweeps and batch size variation.
- Add cross-run comparison artifacts (e.g., run-to-run disagreement between saved predictions).
- Introduce CI thresholds (fail/pass gates) on stability metrics (implemented in Week 2 planning).
- Begin CPU vs GPU and FP32 vs AMP comparisons (where available).